



Forms in React

Introduction to Forms in React

Forms are an essential part of user interfaces, and React provides tools to handle them efficiently. In React, managing form elements like input fields, text areas, checkboxes, radio buttons, and select dropdowns is done using **state** and event handlers.

In React, forms are controlled components, meaning the form data is handled by the **React component** rather than the DOM. The form elements like `<input>`, `<textarea>`, and `<select>` are **controlled** because their values are controlled by React's state.

Controlled Components

A **controlled component** is an input element whose value is controlled by **React state**.

For example, an input element in a React form has its value set to a state variable, and any changes to the input value update the state. React keeps track of the form data through the component's state.

Setting up the React Form

To start with forms in React, we need to:

1. Create a component.
2. Add a form element inside the component.
3. Create state variables to store form data.
4. Bind the state to the form fields.



IOTA ACADEMY

Become Job Ready...

5. Add event handlers to manage the state updates.

Let's start by creating a basic form with an input field and a submit button.

```
import React, { useState } from 'react';

function MyForm() {
  const [name, setName] = useState(''); // state to store the name
  value

  function handleChange(event) {
    setName(event.target.value); // update the state when the input
    changes
  }

  function handleSubmit(event) {
    event.preventDefault(); // prevent default form submission
    behavior
    alert('Form submitted with name: ' + name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Name:
        <input
          type="text"
          value={name} // controlled input with value bound to state
```



```
        onChange={handleChange} // updates state on input change
      />
    </label>
    <button type="submit">Submit</button>
  </form>
);
}

export default MyForm;
```

Explanation of the Code

Let's break down the code:

1. State Declaration:

- `const [name, setName] = useState('');`
 - `name` holds the value of the input field, and `setName` updates this value.

2. Input Element:

- `<input type="text" value={name} onChange={handleChange} />`
 - `value={name}` makes the input field a controlled component. It's connected to the `name` state.
 - `onChange={handleChange}` updates the state whenever the input value changes.

3. Event Handling:



- `handleChange(event)`: This function is triggered whenever the user types into the input. It updates the state with the new value from the input field using `setName`.
- `handleSubmit(event)`: This function handles form submission. We use `event.preventDefault()` to prevent the default behavior of the form (which would reload the page). Instead, we show an alert with the value entered in the form.

Handling Multiple Form Inputs

For more complex forms with multiple fields, each input can be controlled in a similar way, with state variables for each form field.

```
import React, { useState } from 'react';
```

```
function MyForm() {
```

```
  const [formData, setFormData] = useState({  
    name: '',  
    email: '',  
  });
```

```
  function handleChange(event) {
```

```
    const { name, value } = event.target;
```

```
    setFormData({
```

```
      ...formData,
```

```
      [name]: value, // dynamically update the state with the field
```

```
      name
```

```
    });
```



IOTA ACADEMY

Become Job Ready...

```
}

function handleSubmit(event) {
  event.preventDefault();
  alert('Form submitted: ' + JSON.stringify(formData));
}

return (
  <form onSubmit={handleSubmit}>
    <label>
      Name:
      <input
        type="text"
        name="name"
        value={formData.name}
        onChange={handleChange}
      />
    </label>
    <label>
      Email:
      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
      />
    </label>
  </form>
);
```



```
        <button type="submit">Submit</button>
      </form>
    );
  }
}
```

export default MyForm;

Explanation:

- We use a single state variable `formData` to store all form data as an object.
- Each input has a `name` attribute. In the `handleChange` function, we dynamically update the corresponding field in the state using `name` (like `name="name"` and `name="email"`).

Handling Checkboxes and Radio Buttons

Check boxes and radio buttons are a bit different because they (don't have a value in the same way) represent a boolean state or a selected value.

Checkboxes use `checked` to track whether they are selected, and radio buttons allow only one option from a group to be selected at a time.

Checkbox Example:

```
import React, { useState } from 'react';

function MyForm() {
  const [isChecked, setIsChecked] = useState(false);
```



IOTA ACADEMY

Become Job Ready...

```
function handleCheckboxChange(event) {  
    setIsChecked(event.target.checked); // true or false based on  
the checkbox state  
}  
  
function handleSubmit(event) {  
    event.preventDefault();  
    alert('Checkbox is ' + (isChecked ? 'checked' : 'unchecked'));  
}  
  
return (  
    <form onSubmit={handleSubmit}>  
        <label>  
            Accept Terms:  
            <input  
                type="checkbox"  
                checked={isChecked}  
                onChange={handleCheckboxChange}  
            />  
        </label>  
        <button type="submit">Submit</button>  
    </form>  
    );  
}  
  
export default MyForm;
```




Explanation:

- `event.target` refers to the `<input type="checkbox">` element.
- `event.target.checked` is a **boolean value** (true if checked, false if unchecked).
- When the checkbox is clicked, React automatically updates `isChecked` to either true or false based on the **current state of the checkbox**.

Handling Multiple Checkboxes (Dynamic State Update)

If there are multiple checkboxes, we store them in an **object**.

```
import React, { useState } from "react";

function MultiCheckboxExample() {
  const [checkedItems, setCheckedItems] = useState({
    apple: false,
    banana: false,
    mango: false
  });

  function handleChange(event) {
    const { name, checked } = event.target;
    setCheckedItems({
      ...checkedItems,
      [name]: checked, // Dynamically update state
    });
  }
}
```




IOTA ACADEMY

Become Job Ready...

```
return (  
  <div>  
    <label>  
      <input type="checkbox" name="apple"  
checked={checkedItems.apple} onChange={handleChange} />  
      Apple  
    </label>  
    <label>  
      <input type="checkbox" name="banana"  
checked={checkedItems.banana} onChange={handleChange} />  
      Banana  
    </label>  
    <label>  
      <input type="checkbox" name="mango"  
checked={checkedItems.mango} onChange={handleChange} />  
      Mango  
    </label>  
    <p>Selected: {Object.keys(checkedItems).filter((key) =>  
checkedItems[key]).join(", ")}</p>  
  </div>  
);  
}  
  
export default MultiCheckboxExample;
```



Explanation:

- The state `checkedItems` stores the checked status of each item.
- `onChange` dynamically updates the state based on which checkbox is clicked.
- `Object.keys(checkedItems).filter((key) => checkedItems[key])` displays the selected items.

Handling Radio Input in React:

How Radio Buttons Work

- **Radio buttons are grouped** using the same name attribute.
- **Only one radio button can be selected** in a group.
- Unlike checkboxes, **checked is determined by the value stored in state.**

```
import React, { useState } from 'react';

function MyForm() {
  const [gender, setGender] = useState('');

  function handleRadioChange(event) {
    setGender(event.target.value); // update the selected radio
    button value
  }

  function handleSubmit(event) {
    event.preventDefault();
    alert('Selected gender: ' + gender);
  }
}
```



IOTA ACADEMY

Become Job Ready...

```
}

return (
  <form onSubmit={handleSubmit}>
    <label>
      Male:
      <input
        type="radio"
        value="male"
        checked={gender === 'male'}
        onChange={handleRadioChange}
      />
    </label>
    <label>
      Female:
      <input
        type="radio"
        value="female"
        checked={gender === 'female'}
        onChange={handleRadioChange}
      />
    </label>
    <button type="submit">Submit</button>
  </form>
);
}
```



```
export default MyForm;
```

Explanation:

- For radio buttons, we bind the checked attribute to the state variable (gender).
- The onChange event updates the selected gender when a user clicks on a radio button.

Step 5: Handling Select Dropdown

For a select dropdown, you can use the same pattern as for input elements.

```
import React, { useState } from 'react';

function MyForm() {
  const [selectedOption, setSelectedOption] = useState('');

  function handleSelectChange(event) {
    setSelectedOption(event.target.value);
  }

  function handleSubmit(event) {
    event.preventDefault();
    alert('Selected option: ' + selectedOption);
  }

  return (
```



IOTA ACADEMY

Become Job Ready...

```
<form onSubmit={handleSubmit}>
  <label>
    Select your favorite color:
    <select value={selectedOption}
onChange={handleSelectChange}>
      <option value="">--Select--</option>
      <option value="red">Red</option>
      <option value="blue">Blue</option>
      <option value="green">Green</option>
    </select>
  </label>
  <button type="submit">Submit</button>
</form>
);
}

export default MyForm;
```

Explanation:

- The value of the `<select>` element is bound to the `selectedOption` state, and the `onChange` handler updates the state whenever the user selects a different option.



Step 6: Validating Form Data

Form validation is an important part of handling forms in React. You can use state to display error messages if the user doesn't fill out the form correctly.

Here's an example of how you might add validation for an email input:

```
import React, { useState } from 'react';

function MyForm() {
  const [email, setEmail] = useState('');
  const [error, setError] = useState('');

  function handleChange(event) {
    setEmail(event.target.value);
  }

  function handleSubmit(event) {
    event.preventDefault();
    if (!email.includes('@')) {
      setError('Please enter a valid email.');
```



IOTA ACADEMY

Become Job Ready...

```
<form onSubmit={handleSubmit}>
  <label>
    Email:
    <input
      type="email"
      value={email}
      onChange={handleChange}
    />
  </label>
  {error && <div style={{ color: 'red' }}>{error}</div>}
  <button type="submit">Submit</button>
</form>
);
}

export default MyForm;
```

Explanation:

- The error state stores any validation messages.
- If the email doesn't contain the @ symbol, we display an error message.



Exercise:

1. **Task:** Create a form with a **text input** and a **submit button**. The input value should be controlled by React state.

Requirements:

- Display the entered name below the form.
- Prevent form submission if the input is empty.

2. **Task:** Create a form with **Name**, **Email**, and **Password** fields.

Requirements:

- Use a **single state object** to store all input values.
- Show a summary of the entered values below the form when submitted.

3. **Task:** Create a form where the user selects their **gender** (Male/Female/Other) using radio buttons.

Requirements:

- Display the selected gender below the form.
- Ensure that only one radio button can be selected at a time.

4. **Task:** Add a **checkbox** asking users to agree to the terms and conditions before submitting the form.

Requirements:

- Disable the **Submit** button if the checkbox is not checked.

5. **Task:** Create a form where the user selects their country from a **dropdown list**.

Requirements:

- Populate the dropdown with at least 5 country options.
- Show the selected country below the form.

6. **Task:** Create a login form with **Email** and **Password** fields.

Requirements:

- The email must be in a valid format (e.g., @gmail.com).
- The password must be at least 6 characters long.
- Show error messages if validation fails.



IOTA ACADEMY

Become Job Ready...

7. **Task:** As the user types their password, show a **password strength indicator** (Weak, Medium, Strong).

Requirements:

- **Weak:** Less than 6 characters
- **Medium:** 6–10 characters
- **Strong:** More than 10 characters

